

See it think.

A language model is a machine that guesses the next word, one token at a time. This wall shows every guess it made, how sure it was, and what it almost said instead.

NFCU · ACI ALL-HANDS
FROM THE IIAT HACLAB, GONZAGA UNIVERSITY · SEPTEMBER 2026

1

The box in the room

Everything on this wall runs on one rented graphics card in Seattle. Questions are logged; first names are optional.

RTX 4000 Ada
ONE GPU · 20 GB MEMORY

Qwen3 8B
THE MODEL · 8 BILLION WEIGHTS

36 layers
EACH TOKEN PASSES THROUGH ALL OF THEM

~35 / s
TOKENS GENERATED PER SECOND, ONE PERSON AT A TIME



2

Your sentence becomes tokens

The model never sees letters or words. Your question is chopped into **tokens**. Common words are one token, rare words are several, and a space belongs to the word after it.

WHAT THE MODEL ACTUALLY READ

How many r 's are in strawberry ?

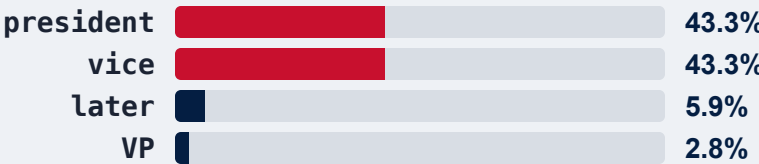
“strawberry” is one token. It has never seen the letters inside it. When it answers “3 r’s” in 2.4 seconds, that is memory, not counting.

3

Every token is a bet

After each token the model gives odds for every possible next one, about 150,000 of them. The wall keeps the top five. Something outside the model has to pick one.

A REAL FORK · “EVERY US PRESIDENT IN ORDER...”



A dead heat. Out of the box the wall takes the top one (**greedy**, temperature 0), so the same question gives the same answer twice. Turn the dials in the wall’s **temperature** chip and it starts rolling dice instead.

4

The answer, colored by confidence

Every token on the wall is tinted by how sure the model was when it chose it (61 tokens, 2.6 seconds). Tap one and its five runners-up appear.

“MONTY HALL, BUT THE HOST OPENS A DOOR AT RANDOM AND IT HAPPENS TO BE A GOAT. SHOULD I SWITCH?”

No, switching doesn't help in this case. The original Monty Hall problem assumes the host always opens a door with a goat, which gives you information. ...

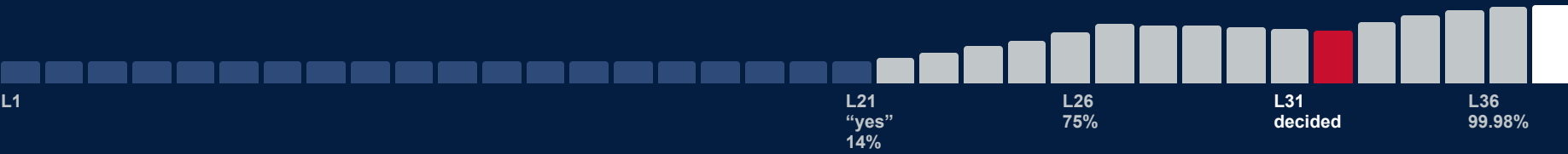
sure · 90% and up unsure · 50–90% coin-flip · under 50%

Nothing here fell to a coin flip, but look where the doubt sits: the verdict “No” at 73% and the hedges **help · in · always** at 61–65%. The facts about the puzzle run at 98–100%. It is sure of the story and less sure of the call.

5

Inside the head, layer by layer

On the lab wall at Gonzaga, a smaller sibling of the model (Qwen3 4B, 36 layers) reads the same question while we watch each layer’s best guess for the first word. Early layers babble. Somewhere in the middle, it makes up its mind.



“No”
the 8B model on the wall · 72.7%

DISAGREE

“Yes”
its 4B sibling · decided at layer 31 of 36 · 0.51 s

Same family, same question, opposite answers. The bigger model is right: when the host opens a door at random, switching is a coin flip. The smaller one settles on the famous answer and never looks back.

6

Why it sometimes loops

Greedy decoding always takes the favorite. After writing “Alright,” the favorite next token is “Alright” again, and each repeat makes the next one likelier. Nothing breaks the cycle. A reasoning model (DeepSeek R1) did exactly this on our server:

Alright, Alright, Alright, Alright, Alright, Alright, Alright, Alright, Alright ... × 10,000 tokens · 281 seconds

temperature 0 · greedy · repeatable 0.6 · rolls weighted dice · escapes the rut

Four live dials on the wall’s **temperature** chip change how the next token is picked. **Temperature** lets the chooser gamble on a runner-up now and then: that is “creativity”, and also “wrong more often”. **Top-p** and **top-k** shorten the list it may pick from. **Repeat penalty** taxes tokens it has already said, which is the direct cure for the loop above. The **leash** (max tokens) is the last brake: the server stops the answer at N tokens no matter what.